Count Inversions

AlgoExpert · Very Hard · Sorting

The problem

- Number of pairs i < j with a[i] > a[j], counted during merge sort.

Intuition

- Merge sort counts inversions: when a right element is placed before remaining left elements, they all form inversions.

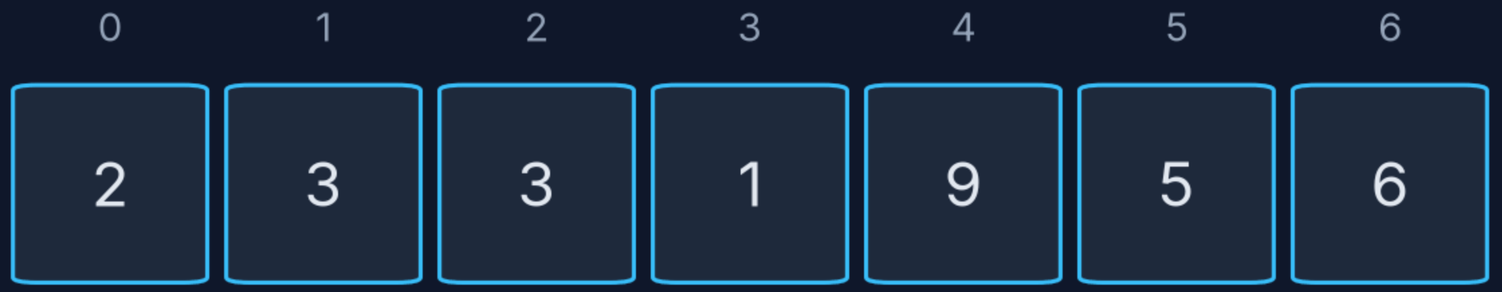
Brute force vs optimal

- Check all pairs → $O(n^2)$
- Merge sort counting → $O(n \log n)$
- BIT over values → $O(n \log n)$

Approach

- 1. Sort halves recursively
- 2. During merge, add remaining left count when taking from the right
- 3. Sum

Worked example

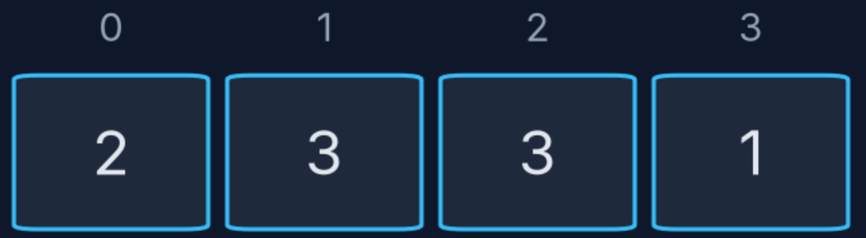



```
inversions: [(0, 3), (1, 3), (2, 3), (4, 5), (4, 6)]
```

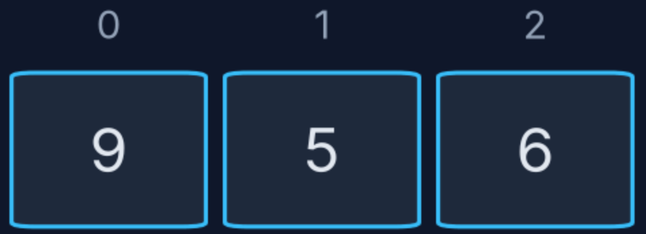
5 inversions

Worked example

left half (sorted: 1,2,3,3)



right half (sorted: 5,6,9)



Merge sort: count within halves recursively

Worked example

```
merging: when right element is taken first,

add (number of remaining left elements)
```

Count cross inversions while merging

Java solution

```java
public class CountInversions {
    public static int countInversions(int[] array) {
        int[] a = array.clone();
        return (int) count(a, new int[a.length], 0, a.length - 1);
    }

    private static long count(int[] a, int[] aux, int lo, int hi) {
        if (lo >= hi) return 0;
        int mid = (lo + hi) >>> 1;
        long inv = count(a, aux, lo, mid) + count(a, aux, mid + 1, hi);
        System.arraycopy(a, lo, aux, lo, hi - lo + 1);
        int i = lo, j = mid + 1;
        for (int k = lo; k <= hi; k++) {
            if (i > mid) a[k] = aux[j++];
            else if (j > hi) a[k] = aux[i++];
            else if (aux[j] < aux[i]) { a[k] = aux[j++]; inv += mid - i + 1; }
            else a[k] = aux[i++];
        }
        return inv;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Edge cases & pitfalls

- Use ≤ to avoid counting equals
- Long for large counts
- Don't mutate input

Interview tips & follow-ups

- Mention Fenwick tree

Recap

- Merge sort counts inversions: when a right element is placed before remaining left elements, they all form inversions.
- Complexity: O(n log n)